Assignment 1

CSA0903-Java Programming

Name: Sowmiya. D

Reg No.:192511063

Scenario — Campus Smart-Parking System

The university campus has a multi-level parking facility used by students, staff and visitors. The facility has several floors, and each floor has a fixed number of numbered slots. Vehicles entering the campus may be two-wheelers, cars or electric vehicles (EVs). EVs may also use charging slots, which are charged separately.

When a vehicle enters, the system must allocate a suitable free slot, issue a parking ticket with a unique ticket number, and record the entry time. When the vehicle leaves, the system must release the slot and compute the parking fee. Fees depend on the vehicle type and the duration of parking. Staff and students holding a valid monthly pass park free of charge, while visitors pay the full fee.

The facility may contain:

1 floor × 20 slots → 3 floors × 50 slots → 5 floors × 100 slots

You are required to analyse the problem, design an object-oriented model for it, implement the model in Java and validate it. Your design must remain correct and easy to extend as the facility and its rules grow.

Design Challenge:

The university now wants to add new vehicle categories and a new pricing rule without rewriting the classes that already work.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 4 – Design Classes, Objects and Constructors 4 Marks

Bloom's Level: K4 – Analyse

Design the classes identified in Task 1 and decide how their objects will be created.

Discuss:

- The fields and methods of each class
- Default, parameterised and overloaded constructors, and when each is used
- Use of the this keyword and constructor chaining to avoid repeated code
- Whether a copy constructor is needed anywhere (for example: re-issuing a duplicate ticket) and why
- How objects are stored in memory (reference variables, stack and heap)

Alternative Concept and Complexity Analysis

You used overloaded constructors and constructor chaining. If objects were instead created with a default constructor followed by setter calls, or through static factory methods, what would the solution be? Compare how each approach guarantees a valid object, and analyse the time and space cost of creating n ParkingTicket objects in each approach.

Evidence Expected

Class definitions with justified constructors, use of this and constructor chaining, and an explanation of object creation.

Solution:

TASK 4 – Design Classes, Objects and Constructors

1. Introduction

The Campus Smart-Parking System uses object-oriented programming to represent parking facilities, vehicles, parking slots, and parking tickets. Each class contains suitable fields, methods, and constructors to manage parking operations.

2. Classes, Fields and Methods

Class	Fields	Methods
ParkingFacility	facilityName, floors	allocateSlot(), releaseSlot()
ParkingSlot	slotNumber, floorNumber, isOccupied	occupy(), release()
Vehicle	vehicleNumber, vehicleType	getVehicleDetails()
ParkingTicket	ticketId, vehicle, entryTime, exitTime, fee	calculateFee(), closeTicket()

3. Constructors

A constructor initializes an object when it is created.

A. Default constructor

A constructor without parameters.

```
class Vehicle {

    String vehicleNumber;

    String vehicleType;


    Vehicle() {
```

```
        vehicleNumber = "UNKNOWN";
        vehicleType = "CAR";
    }
}
```

Use: Creates an object with default values.

B. Parameterised constructor

A constructor that accepts parameters.

```
class Vehicle {
    String vehicleNumber;
    String vehicleType;

    Vehicle(String vehicleNumber, String vehicleType) {
        this.vehicleNumber = vehicleNumber;
        this.vehicleType = vehicleType;
    }
}
```

Use: Creates a vehicle with specific details.

C. Overloaded constructors

A class can have multiple constructors with different parameter lists.

```
class ParkingTicket {
    int ticketId;
    String vehicleNumber;

    ParkingTicket() {
        this(0, "UNKNOWN");
    }

    ParkingTicket(int ticketId) {
```

```
        this(ticketId, "UNKNOWN");
    }

    ParkingTicket(int ticketId, String vehicleNumber) {
        this.ticketId = ticketId;
        this.vehicleNumber = vehicleNumber;
    }
}
```

Use: Allows tickets to be created with different levels of information.

4. Use of this Keyword and Constructor Chaining

The this keyword refers to the current object. It helps distinguish instance variables from constructor parameters.

Example:

```
this.ticketId = ticketId;
```

Constructor chaining means calling one constructor from another using this().

Example:

```
ParkingTicket() {
    this(0, "UNKNOWN");
}
```

Benefits:

- Avoids repeated initialization code.
- Improves code reusability.
- Makes constructors easier to maintain.
- Ensures consistent object initialization.

5. Copy Constructor

A copy constructor creates a new object using the values of an existing object.

Example:

```
class ParkingTicket {
    int ticketId;
```

```
    String vehicleNumber;


    ParkingTicket(int ticketId, String vehicleNumber) {
        this.ticketId = ticketId;
        this.vehicleNumber = vehicleNumber;
    }


    ParkingTicket(ParkingTicket other) {
        this(other.ticketId, other.vehicleNumber);
    }
}
```

Use in the parking system:

A copy constructor can be useful when creating a duplicate ticket containing the same vehicle details. However, a re-issued ticket should normally receive a new unique ticket ID and a new issue time. Therefore, copying every field directly may not be appropriate.

6. Object Creation and Memory Allocation

In Java, objects are generally created using the new keyword.

Example:

ParkingTicket ticket1 = new ParkingTicket(101, "TN01AB1234");

Memory allocation:

- Stack: Stores local variables and reference variables such as ticket1.
- Heap: Stores the actual ParkingTicket object created using new.
- Reference variable: Holds the reference to the object in memory.

When an object is no longer reachable, Java's garbage collector can reclaim its memory.

Understanding Stack and Heap Memory in Java: Detailed Explanation with Examples and Use Cases | by Vinotech | Medium

medium.com

7. Alternative Concept and Complexity Analysis

Three approaches can be used to create ParkingTicket objects.

A. Default constructor followed by setter methods

```
ParkingTicket ticket = new ParkingTicket();
ticket.setTicketId(101);
ticket.setVehicleNumber("TN01AB1234");
```

- Easy to create and modify objects.
- May temporarily create an incomplete or invalid object.
- Validation must be performed in setter methods or before using the object.

B. Parameterised constructor with constructor chaining

```
ParkingTicket ticket =
    new ParkingTicket(101, "TN01AB1234");
```

- Initializes all required fields together.
- Helps ensure the object is valid at creation.
- Constructor chaining avoids repeated initialization code.

C. Static factory method

A static factory method creates and returns an object.

```
class ParkingTicket {
    int ticketId;
    String vehicleNumber;
    private ParkingTicket(int ticketId, String vehicleNumber) {
        this.ticketId = ticketId;
        this.vehicleNumber = vehicleNumber;
    }
    static ParkingTicket createTicket(int id, String number) {
        if (id <= 0 || number == null || number.isBlank()) {
            throw new IllegalArgumentException("Invalid ticket details");
        }
        return new ParkingTicket(id, number);
    }
}
```

Use:

ParkingTicket ticket =

ParkingTicket.createTicket(101, "TN01AB1234");

- Provides better control over object creation.
- Can validate data before returning the object.
- Can return different object types or reuse existing objects when appropriate.

8. Time and Space Complexity

Let n be the number of ParkingTicket objects created.

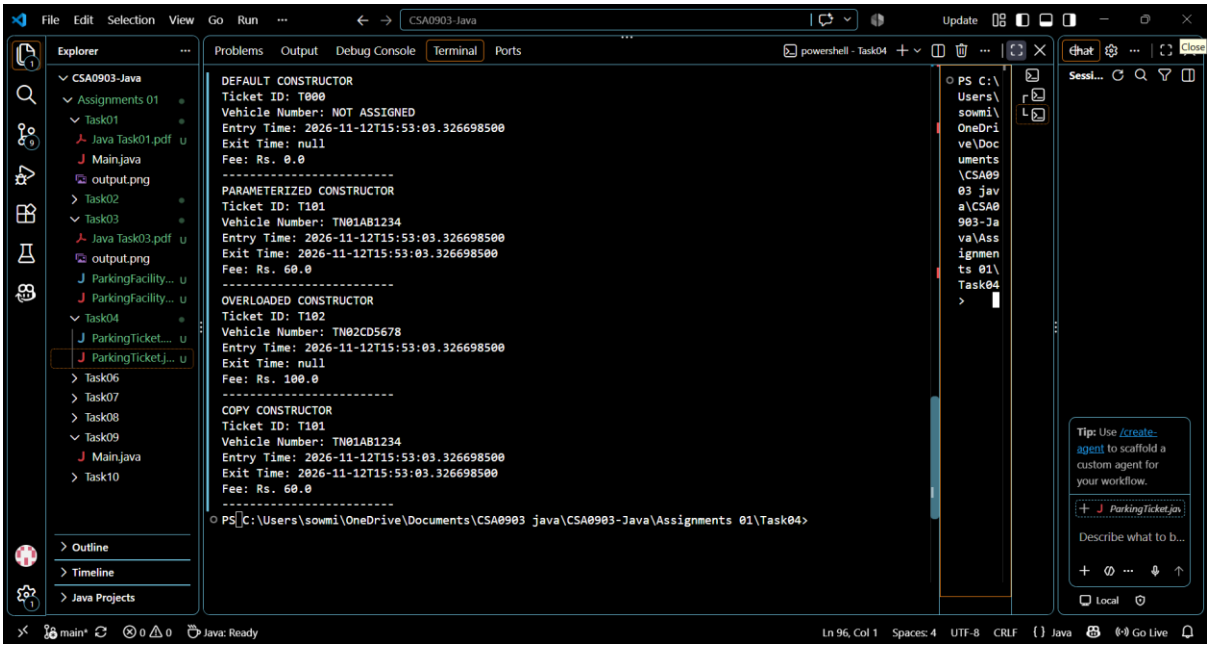
Approach	**Time complexity**	**Space complexity**
Default constructor + setters	O(n)	O(n)
Parameterised constructor	O(n)	O(n)
Static factory method	O(n)	O(n)

Explanation:

- Creating one ticket takes constant time, O(1), assuming a fixed number of fields and simple validation.
- Creating n tickets takes O(n) time.
- Storing n ticket objects requires O(n) space.
- All three approaches have the same asymptotic complexity, although validation and method-call overhead may differ.

9. Conclusion

The Campus Smart-Parking System uses classes and objects to represent real-world parking entities. Parameterised and overloaded constructors help initialize objects with valid details. Constructor chaining and the this keyword reduce repeated code. Copy constructors can be useful for duplicating ticket information, while static factory methods provide additional control over object creation.


File Edit Selection View Go Run
CSA0903-Java
Update
Explorer
CSA0903-Java
Assignments 01
Task01
Java Task01.pdf
Main.java
output.png
Task02
Task03
Java Task03.pdf
output.png
ParkingFacility...
ParkingFacility...
Task04
ParkingTicket....
ParkingTicket.j...
Task06
Task07
Task08
Task09
Main.java
Task10
Outline
Timeline
Java Projects
Problems Output Debug Console Terminal Ports
powershell - Task04
DEFAULT CONSTRUCTOR
Ticket ID: T000
Vehicle Number: NOT ASSIGNED
Entry Time: 2026-11-12T15:53:03.326698500
Exit Time: null
Fee: Rs. 0.0
PARAMETERIZED CONSTRUCTOR
Ticket ID: T101
Vehicle Number: TN01AB1234
Entry Time: 2026-11-12T15:53:03.326698500
Exit Time: 2026-11-12T15:53:03.326698500
Fee: Rs. 60.0
OVERLOADED CONSTRUCTOR
Ticket ID: T102
Vehicle Number: TN02CD5678
Entry Time: 2026-11-12T15:53:03.326698500
Exit Time: null
Fee: Rs. 100.0
COPY CONSTRUCTOR
Ticket ID: T101
Vehicle Number: TN01AB1234
Entry Time: 2026-11-12T15:53:03.326698500
Exit Time: 2026-11-12T15:53:03.326698500
Fee: Rs. 60.0
PS C:\Users\sowmi\OneDrive\Documents\CSA0903 java\CSA0903-Java\Assignments 01\Task04>
PS C:\Users\sowmi\OneDrive\Documents\CSA0903 java\CSA0903-Java\Assignments 01\Task04>
chat
Close
Sessi...
Tip: Use /create-agent to scaffold a custom agent for your workflow.
ParkingTicket.jav
Describe what to b...
Local
main*
0 0
Java: Ready
Ln 96, Col 1
Spaces: 4
UTF-8
CRLF
Java
Go Live